18-740: Modern Computer Architecture & Design

Lecture 5:
"Register Renaming & Out-of-Order Execution"

John Paul Shen

September 8, 2026

- Required Reading Assignment:
  - **Chapter 5 of Shen and Lipasti (SnL).**
- Recommended References:
  - Robert Tomasulo, "An Efficient Algorithm for Exploiting Multiple Arithmetic Units," IBM Journal of Research and Development, 11(1):25-33, January 1967.

Electrical & Computer ENGINEERING

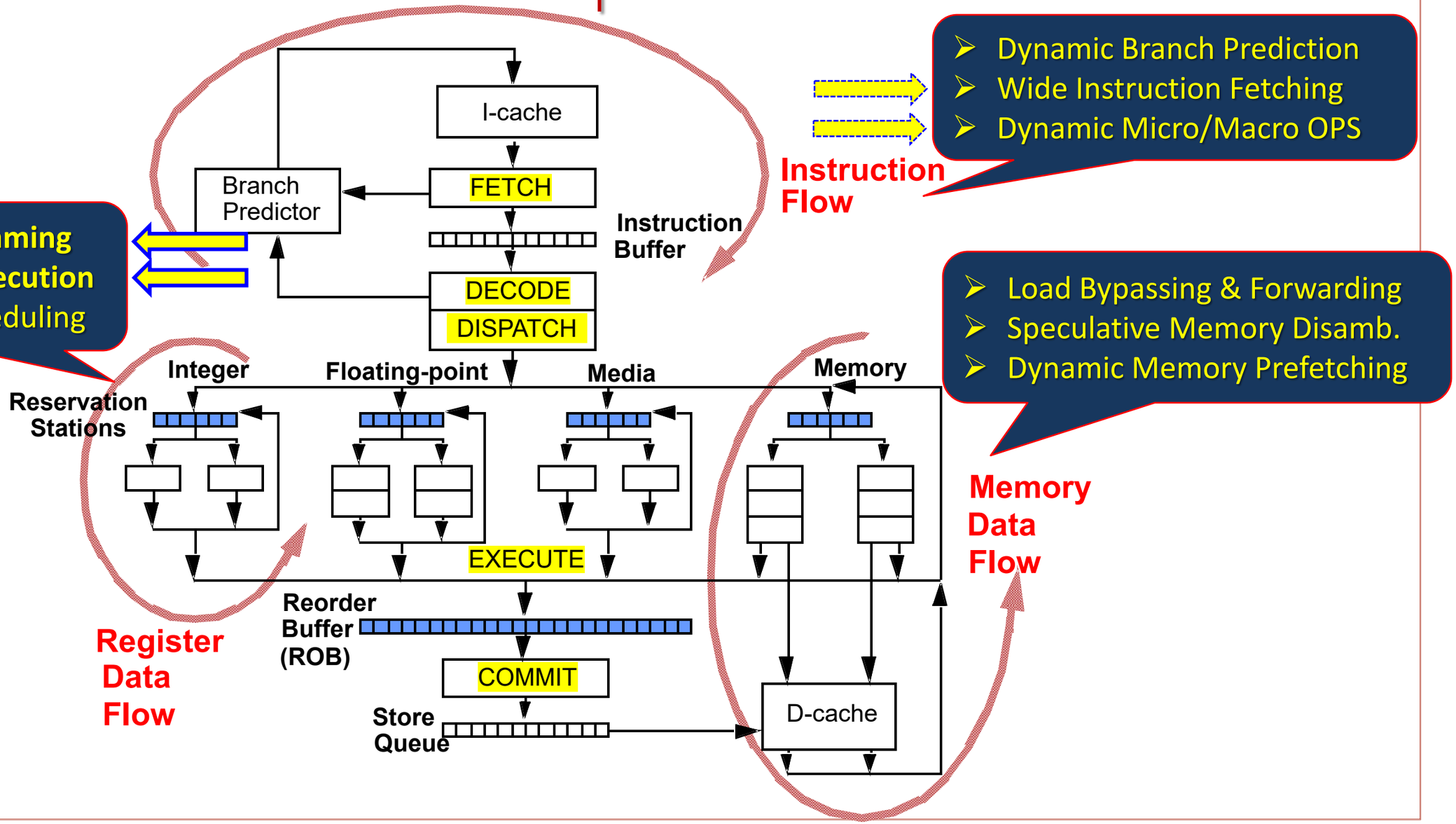
18-740: Modern Computer Architecture & Design

Lecture 5:
"Register Renaming & Out-of-Order Execution"

A. Register Renaming Techniques
B. Out-of-Order Instruction Execution
C. Tomasulo's Algorithm (historical reference)

Electrical & Computer ENGINEERING

Three Flow Paths of Superscalar Processors



- Dynamic Branch Prediction
- Wide Instruction Fetching
- Dynamic Micro/Macro OPS

- **Register Renaming**
- **Data Flow Execution**
- Dynamic Scheduling

- Load Bypassing & Forwarding
- Speculative Memory Disamb.
- Dynamic Memory Prefetching

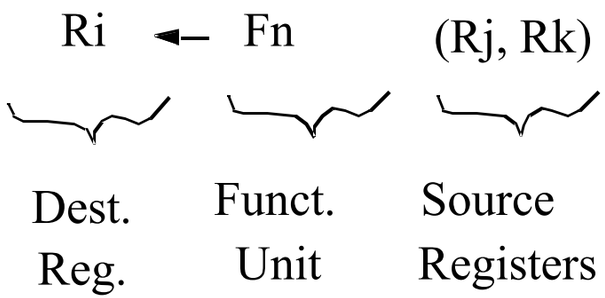
A. Register Renaming Techniques



Electrical & Computer ENGINEERING

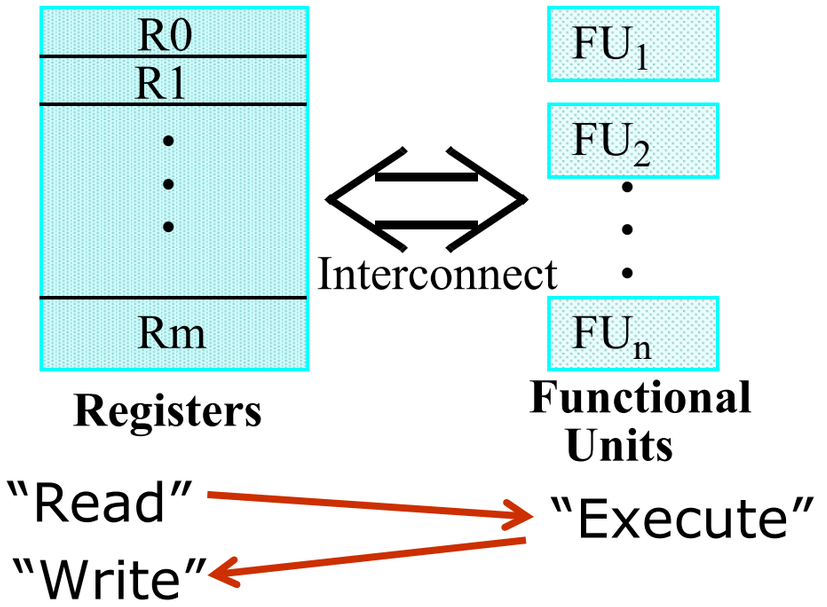
Register Data Flow

Each ALU Instruction:



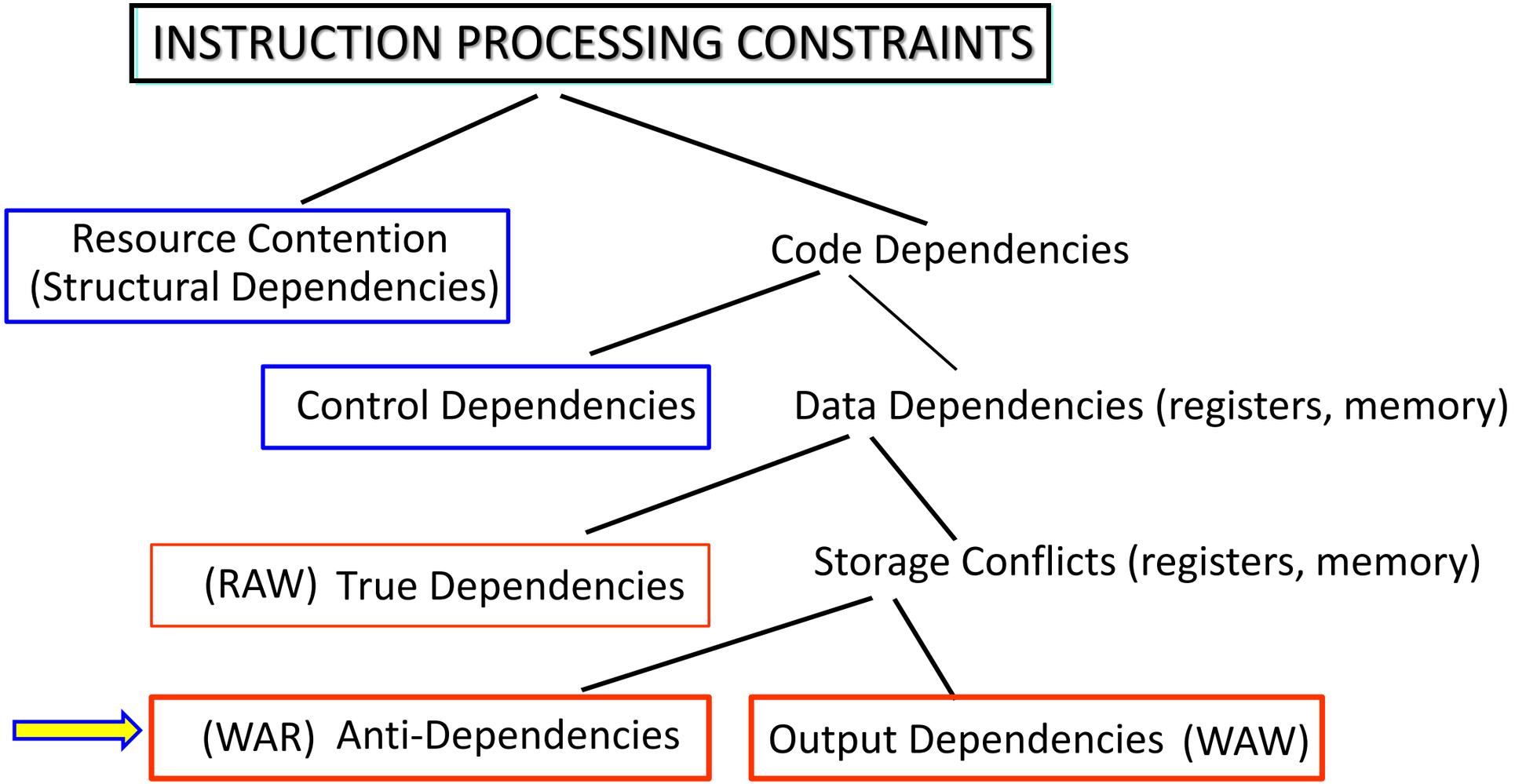
"Register Transfer"

INSTRUCTION EXECUTION MODEL

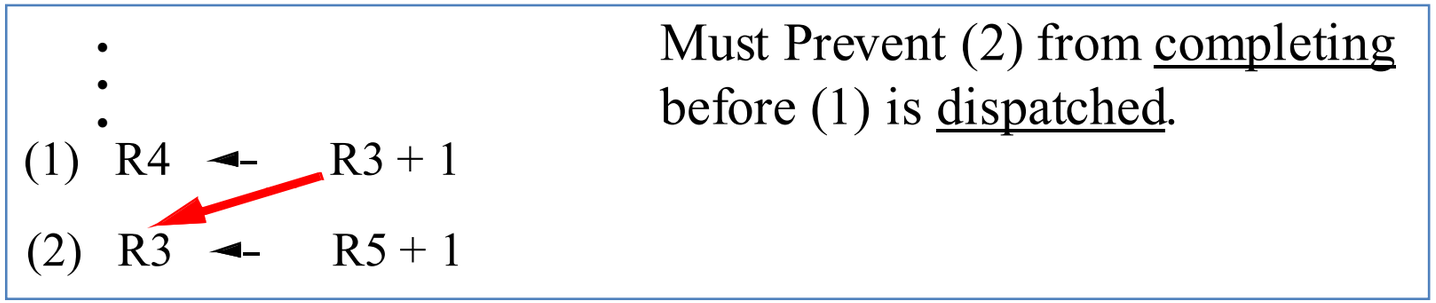


- For an instruction to execute:
  - Need availability of functional unit Fn (structural dependency)
  - Need availability of Rj and Rk (RAW: true data dependency)
  - Need availability of Ri (WAR and WAW: anti and output dependencies)

The Big Picture: Impediments Limiting ILP



Resolving Anti-Dependencies



STALL DISPATCHING

DELAY DISPATCHING OF (2)

REQUIRE RECHECKING AND REACCESSING

WAR only

COPY OPERAND

COPY NOT-YET-USED OPERAND TO PREVENT BEING OVERWRITTEN

MUST USE TAG IF ACTUAL OPERAND NOT-YET-AVAILABLE

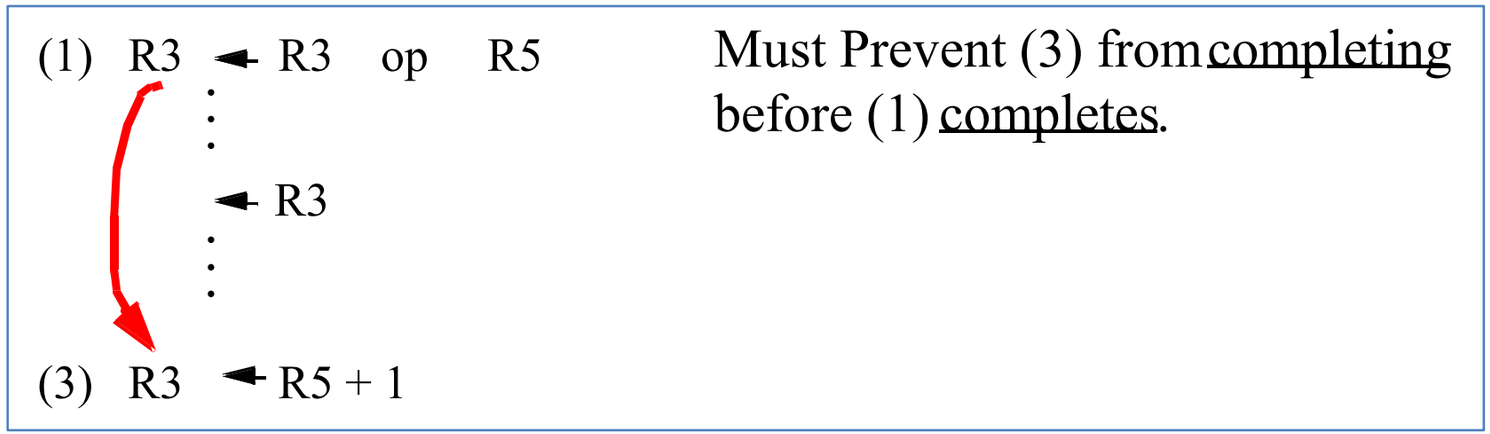
WAR and WAW

RENAME REGISTER

HARDWARE ALLOCATION

```
R3   ← …
     ← R3
R3'  ← …
     ← R3'
```

Resolving Output Dependencies



Must Prevent (3) from completing before (1) completes.

STALL DISPATCHING/ISSUING

DENOTE OUTPUT DEPENDENCY

HOLD DISPATCHING UNTIL RESOLUTION OF DEPENDENCY

ALLOW DECODING OF SUBSEQUENT INSTRUCTIONS

RENAME REGISTER

HARDWARE ALLOCATION

```
R3   ← …
     ← R3
R3'  ← …
     ← R3'
```

Causes of Register Storage Conflict

REGISTER RECYCLING

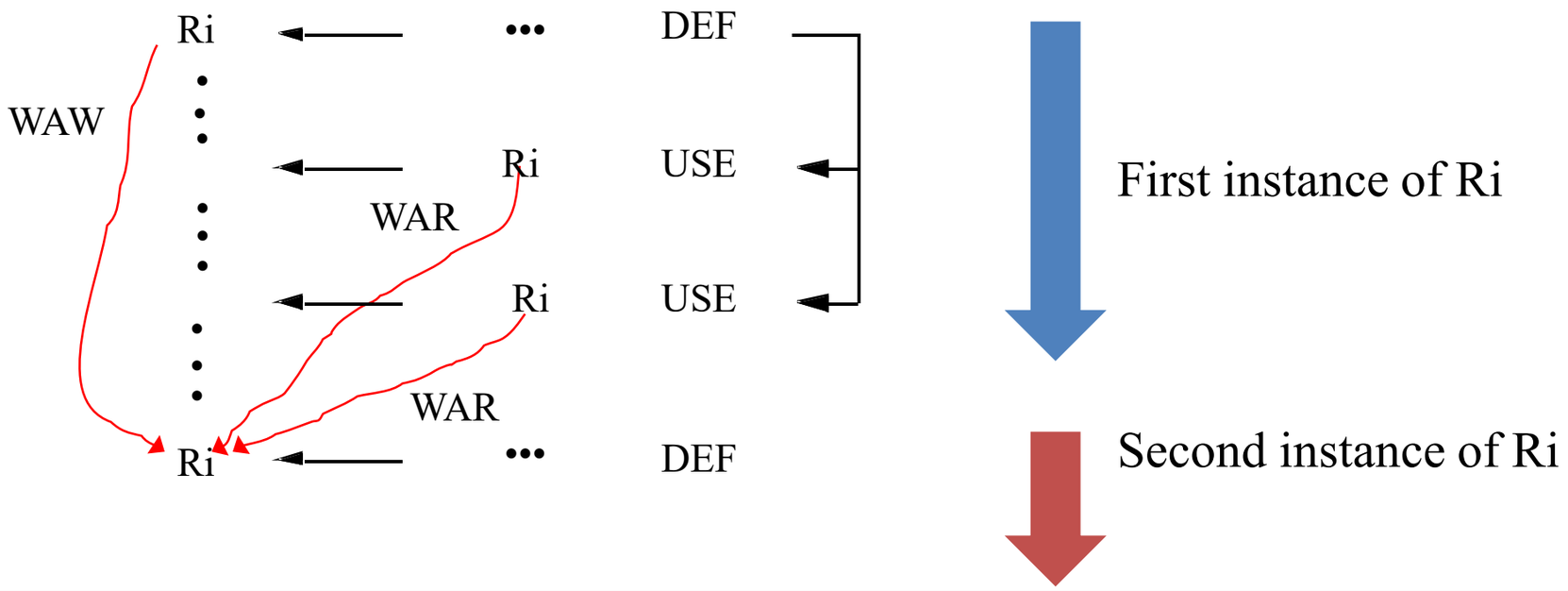
MAXIMIZE USE OF REGISTERS

MULTIPLE ASSIGNMENTS OF VALUES TO REGISTERS

OUT OF ORDER ISSUING AND COMPLETION

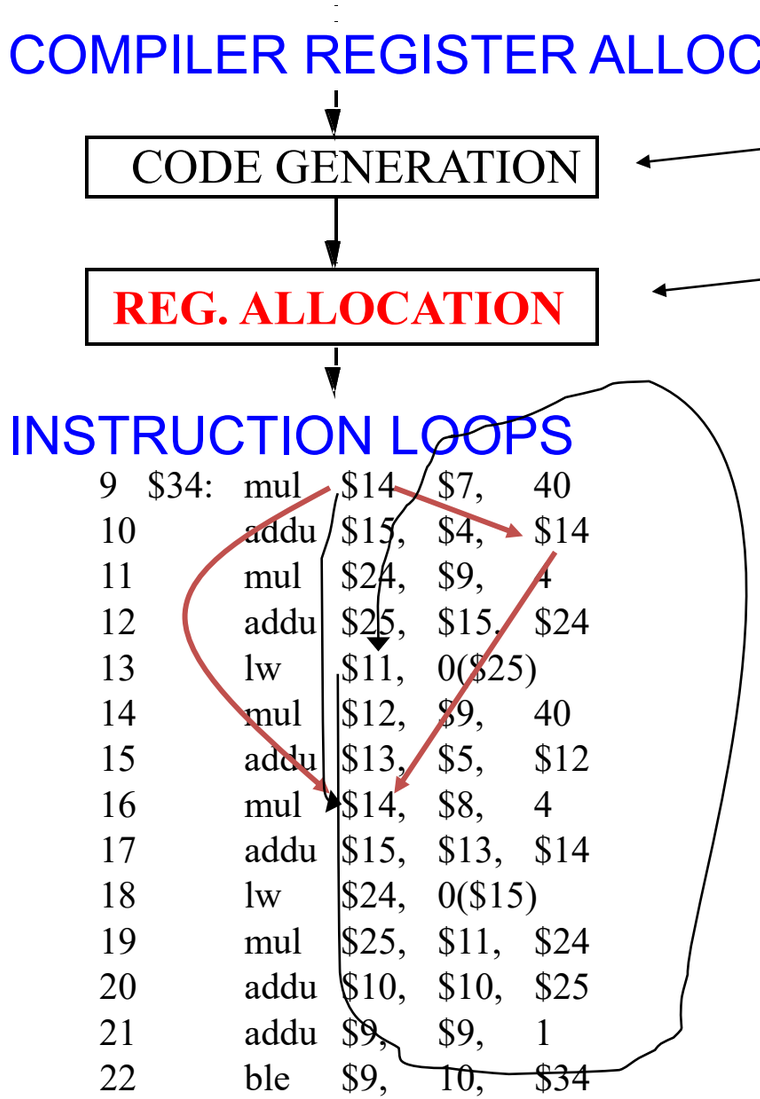
LOSE IMPLIED PRECEDENCE OF SEQUENTIAL CODE

LOSE 1-1 CORRESPONDENCE BETWEEN VALUES AND REGISTERS



Reason for WAW and WAR: Register Recycling

- Intermediate code
  - Infinite number of symbolic registers
  - One used per value definition
- Register Allocation via graph coloring
  - Map symbolic registers to few architectural registers
  - Leads to register reuses



Single Assignment, Symbolic Reg.

Map Symbolic Reg. to "Physical Reg."
Maximize Reuse of Reg.

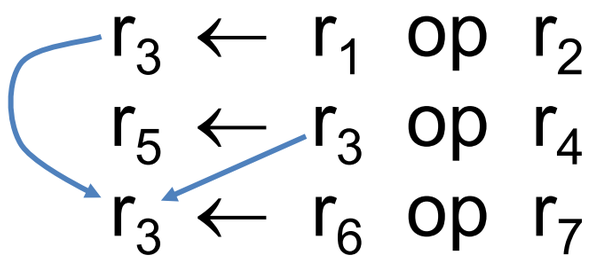
"Spill code" (if not enough registers)

```
For (k=1;k<= 10; k++)
   t += a [i] [k] * b [k] [j] ;
```

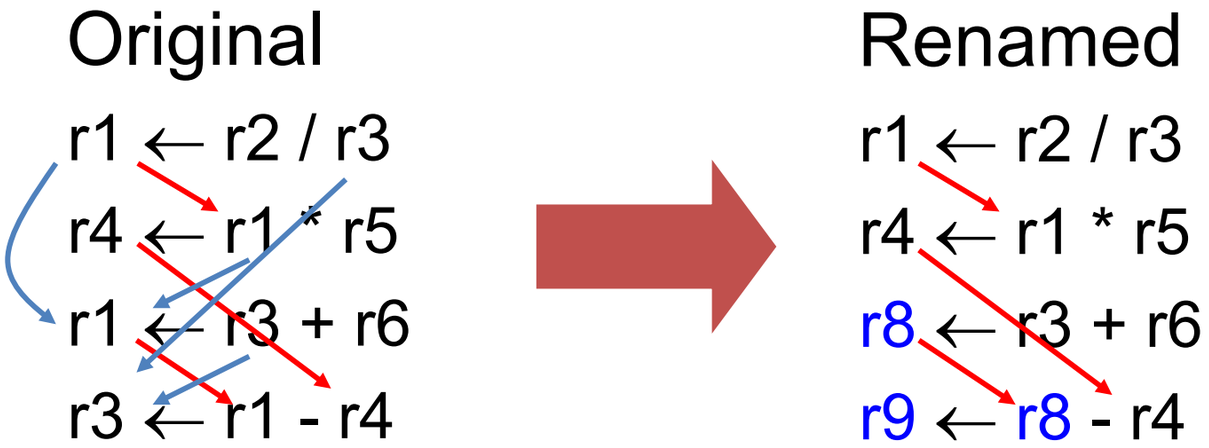
- Dynamic register reuse
  - Reuse same set of registers in each iteration
  - Overlapped execution of multiple iterations

Register Renaming: The Idea

- Anti and output dependencies are false dependencies



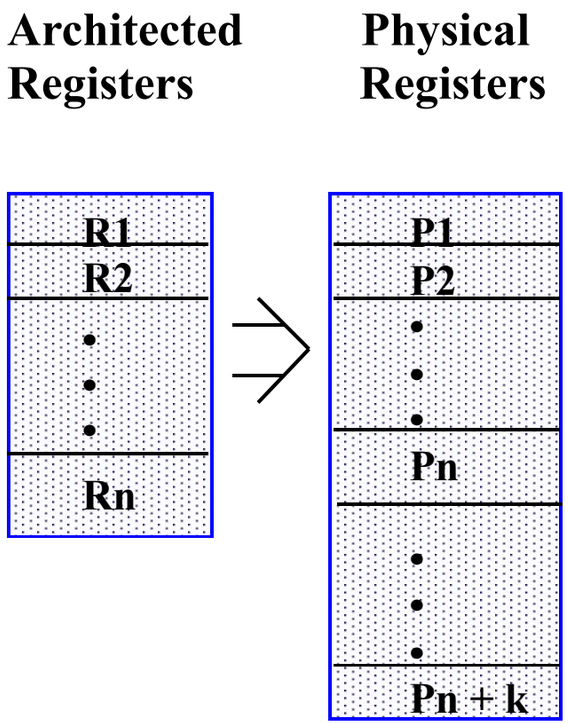
- The dependency is on name/location rather than data
- Given unlimited number of registers, anti and output dependencies can always be eliminated



Register Renaming

Register Renaming Resolves:

- **Anti-Dependencies**
- **Output Dependencies**



Design of Redundant Registers

Number:

- **One**
- **Multiple**

Allocation:

- **Fixed for Each Register**
- **Pooled for all Regsiters**

Location:

- **Attached to Register File (Centralized)**
- **Attached to functional units (Distributed)**

Elements of Register Renaming

A pool of extra registers

- Managed as temporary, single-assignment registers
  (eliminates WAW and WAR)
- Logically separate from architectural registers defined in the ISA
- Many physical organizations are possible

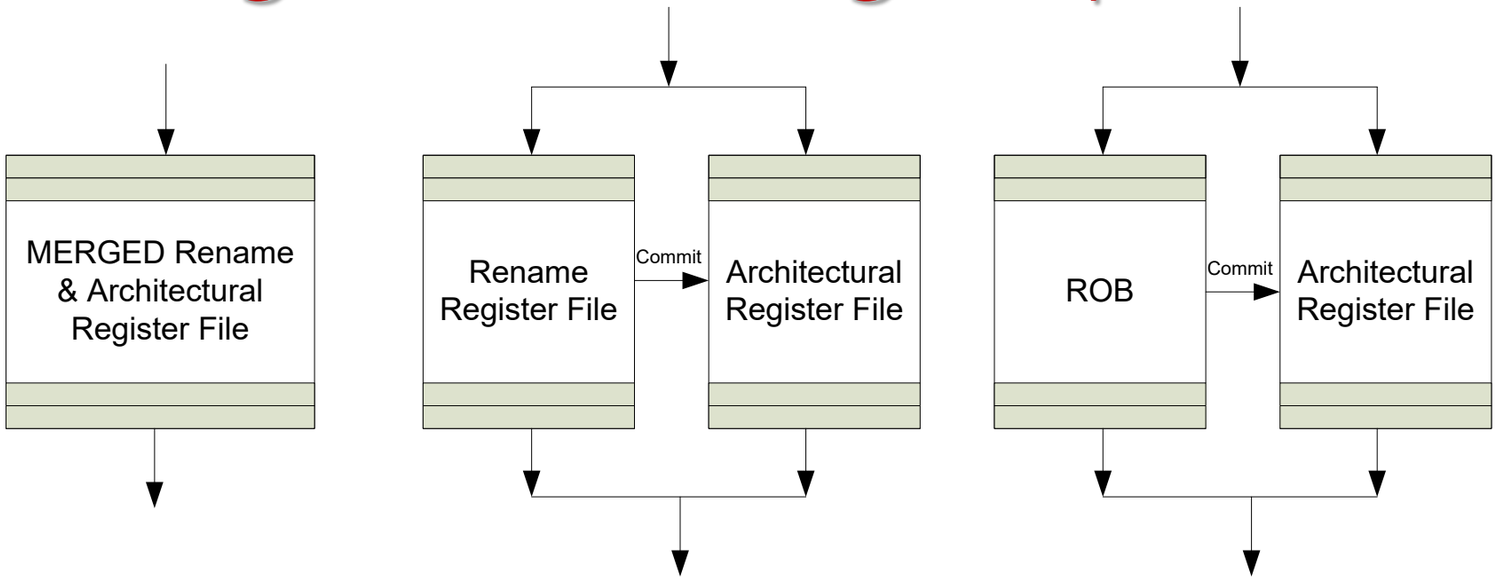
An allocation and mapping mechanism

- Given an architectural source register name, where is its current definition
  (value, location, ready or not)
- Given an architectural destination register name,
  1. How to find an unused rename register
  2. How to establish a new mapping for later references
- When to reclaim a mapping? when to reclaim a register?

Register Renaming Implementation

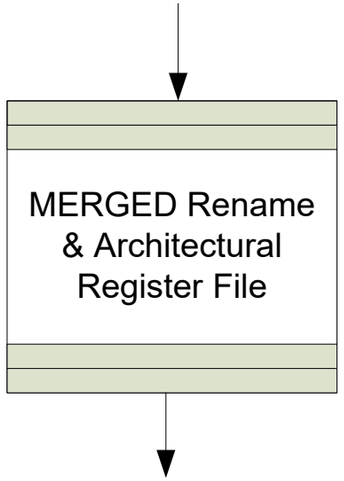
- Renaming:
  - Map small set of architectural registers to a large set of physical registers
  - New mapping for an architectural register when it is assigned a new value
- Renaming buffer organization (how are registers stored)
  - Unified RF (PRF), split RFs (ARF + RRF), renaming in the ROB (ARF + ROB)
  - RF: register file; PRF: physical RF; ARF: architectural RF; RRF: rename RF;
- Number of renaming registers
- Number of read/write ports needed
- Register mapping (how do I find the register I am looking for)

Renaming Buffer Design Options

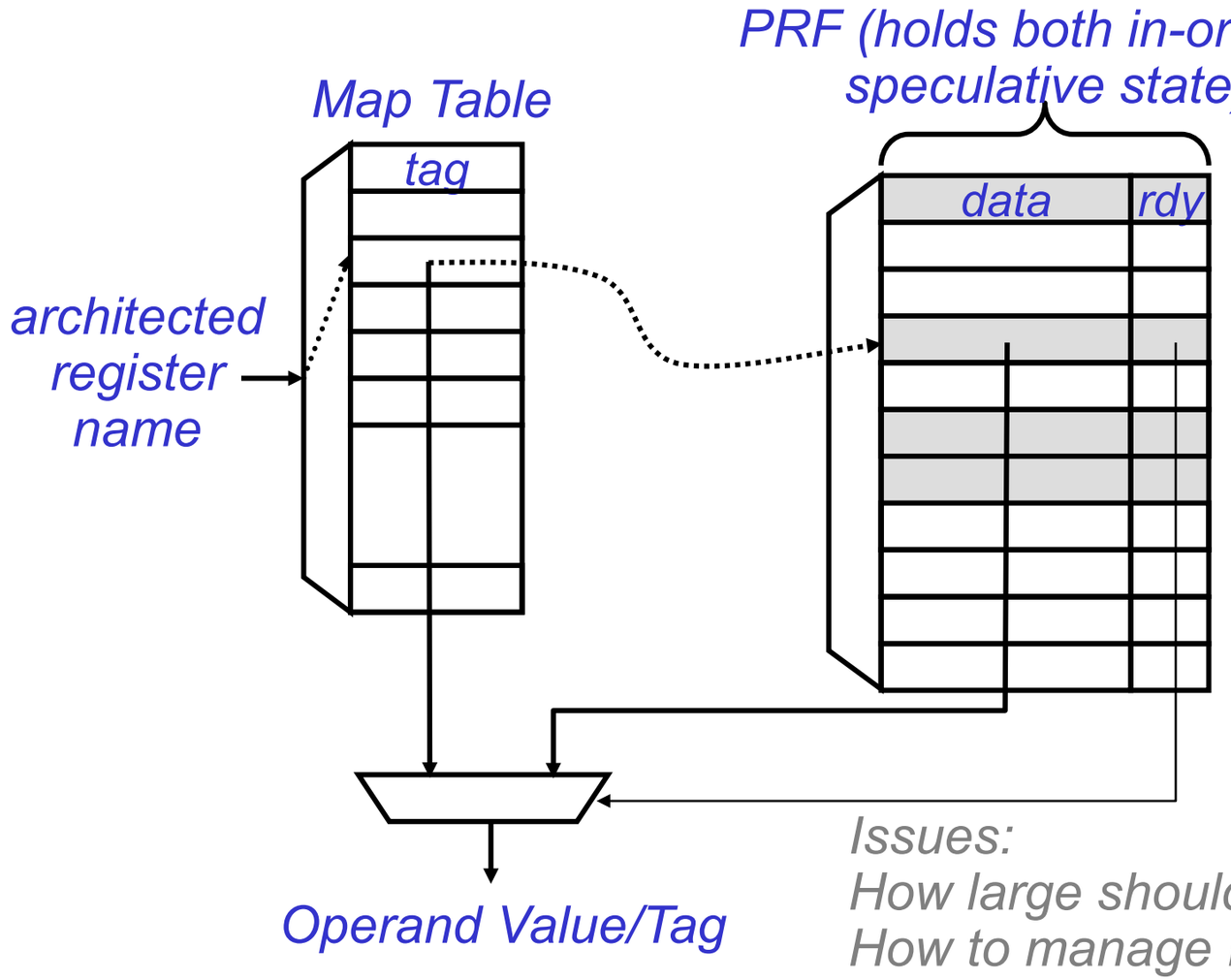


- Unified/merged physical register file – MIPS R10K, Alpha 21264 (PRF)
  - Registers change role between architecture to renamed
- Rename register file (RRF) – PA 8500, PPC 620 (RRF + ARF)
  - Holds new values until they are committed to ARF (extra transfer)
- Renaming in the ROB – Pentium III (ROB + ARF)
- Note: can have a single scheme or separate for integer/FP

Unified/Merged Physical Register File (PRF)

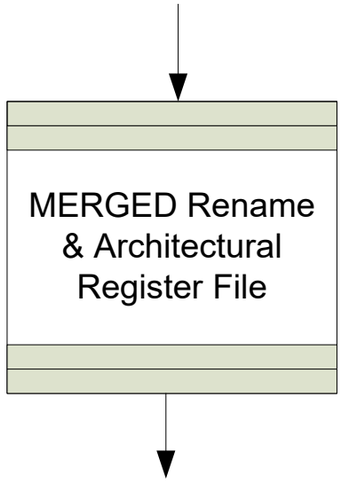


MIPS R10000,
DEC Alpha 21264



Issues:
How large should the PRF be?
How to manage PRF?
How to recover the Map Table on br. misprediction & exceptions?

Allocation & Deallocation of Unified PRF



MIPS R10000, DEC Alpha 21264

When speculative register state is committed, no copying is required

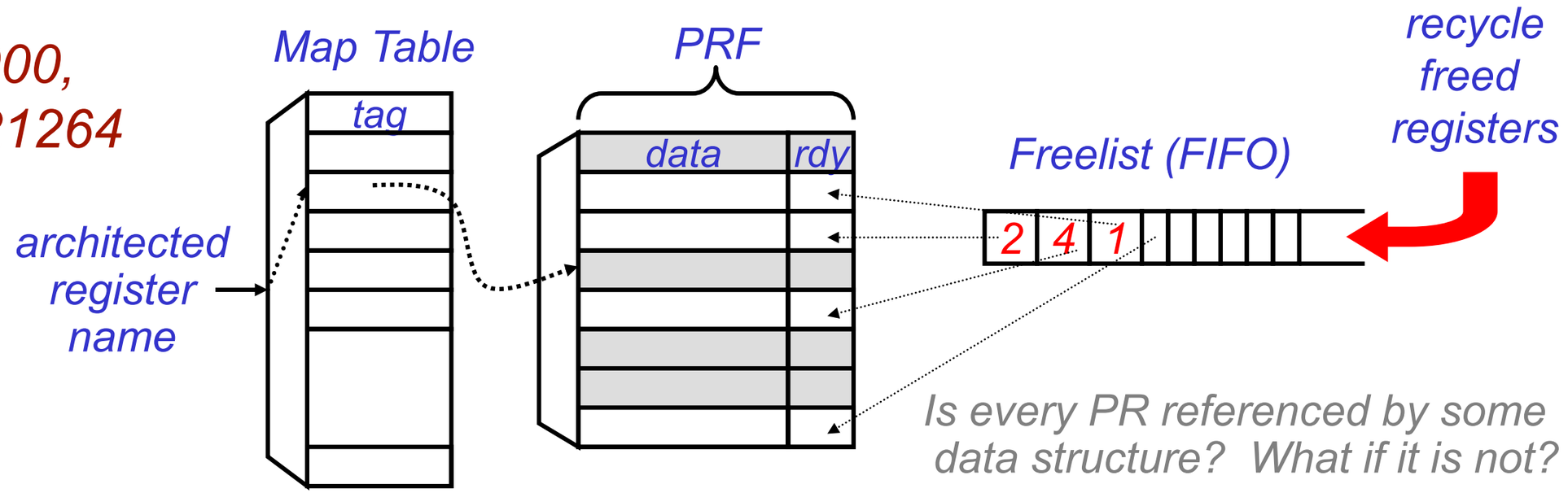
(in fact, not much happens)

PRF registers are not de/allocated in order *Why?*

Freelist - stores a pool of unused registers

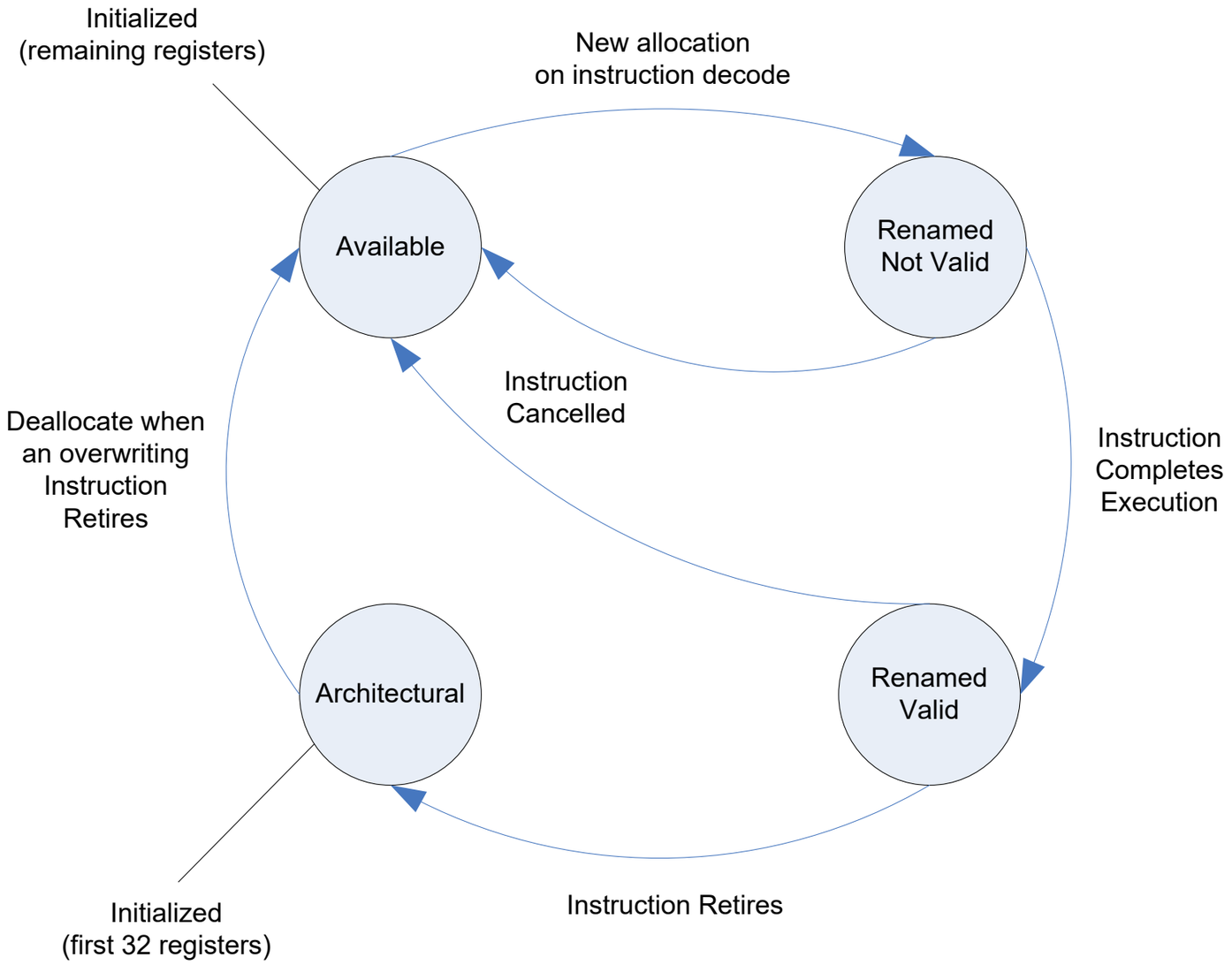
When to recycle a physical register?

i.e. when do we know a register is no longer referred to (by logical name or by physical name)

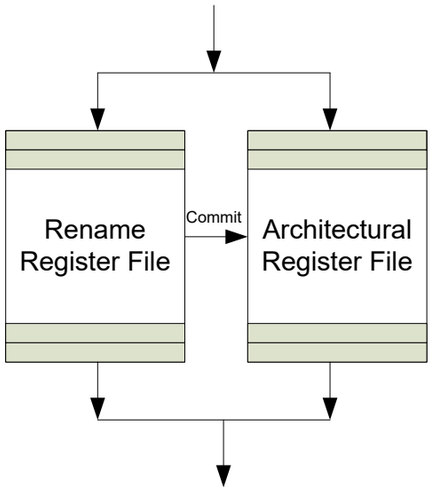


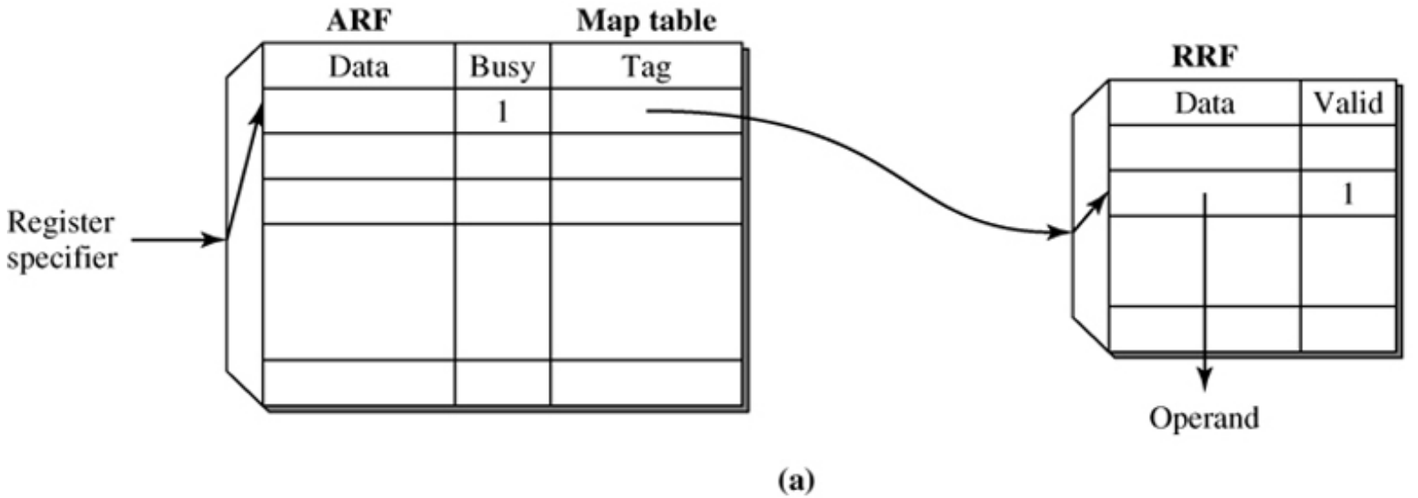
Is every PR referenced by some data structure? What if it is not?

Unified Register File: Physical Register FSM

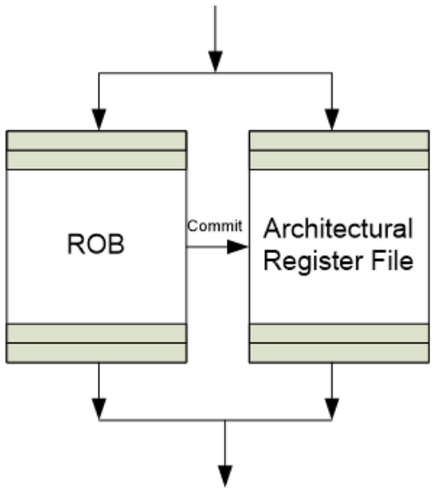


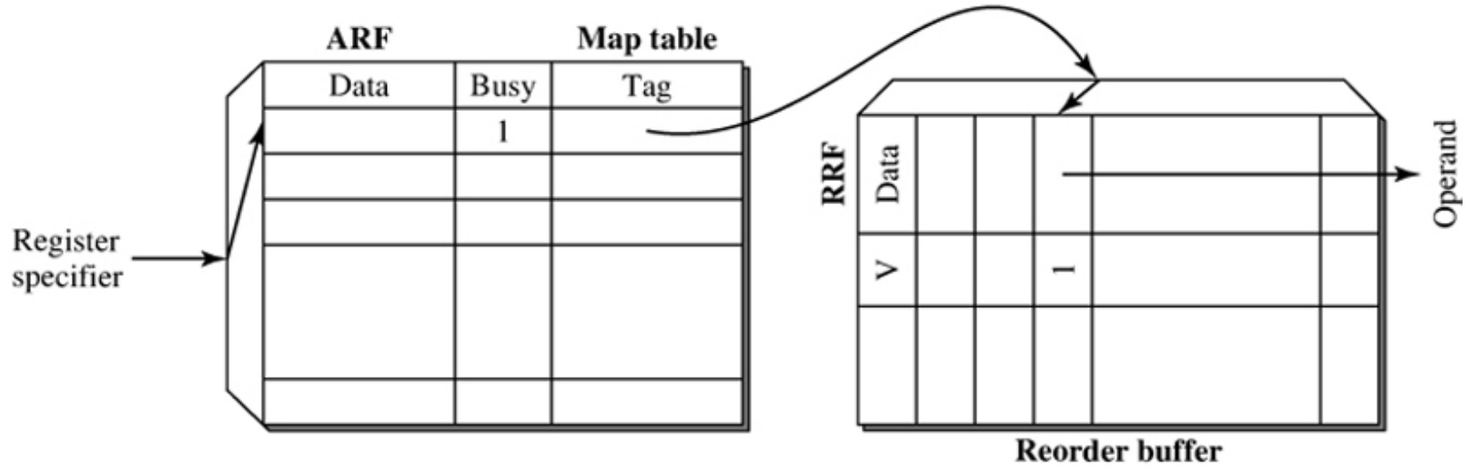
Integrating Map Tables with the ARF





(a)





(b)

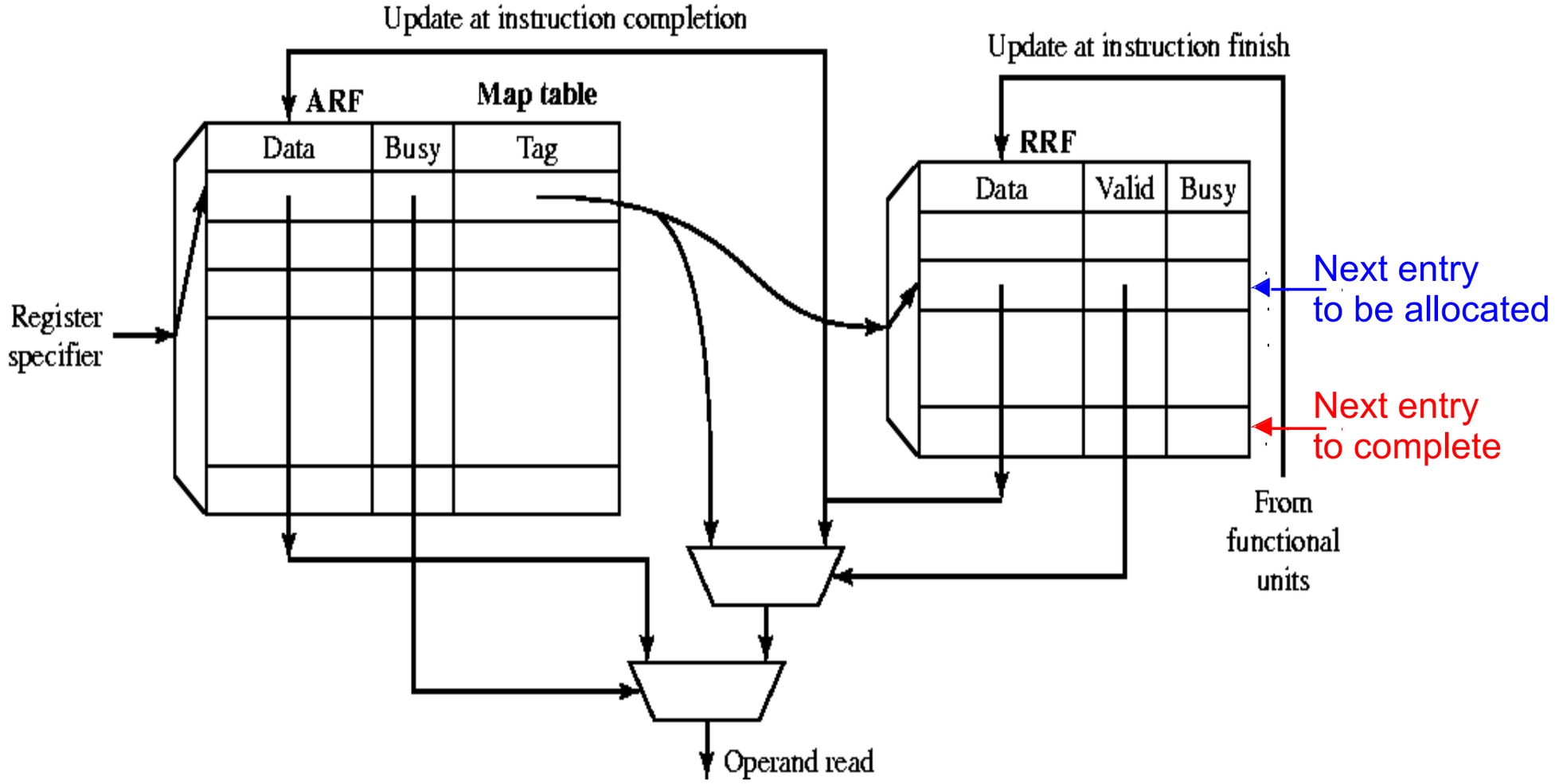
Register Renaming Operations: Allocation, Lookup, De-Allocation

- At Decode/Dispatch: for each instruction
  - Check source registers for availability of source operands
  - Map destination register to new physical register
    - Stall if no register available
  - Note: must have enough ports to map tables
- At Finish: update physical register in RRF or ROB
- At Complete: update architectural register
  - Copy from RRF/ROB to ARF and De-allocate RRF/ROB entry

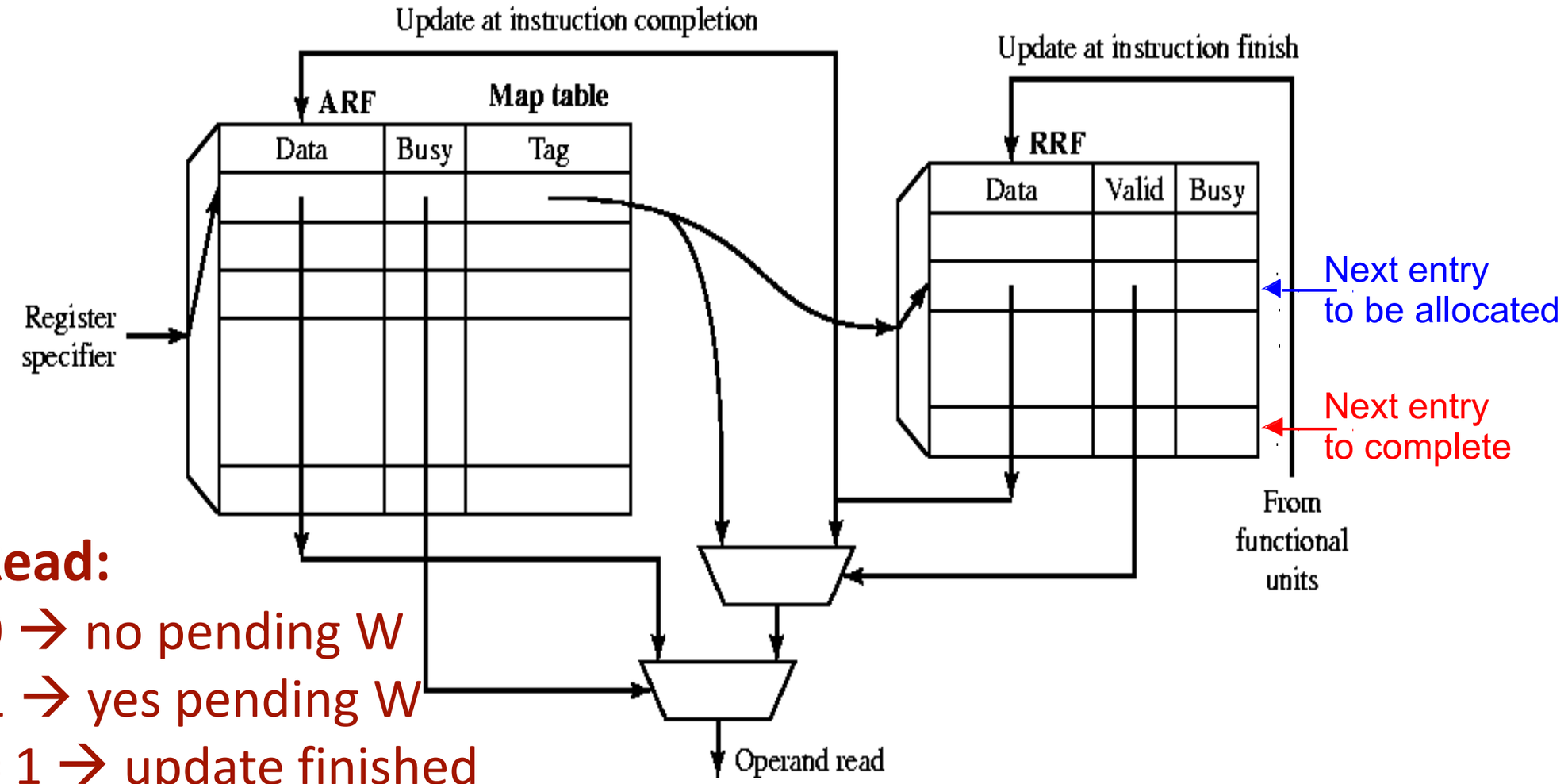
- Question: can we allocate later or deallocate earlier?

Register Renaming Tasks

- Source Read, Destination Allocate, Register Update



Register Renaming Tasks – Source Read



1. Source Read:

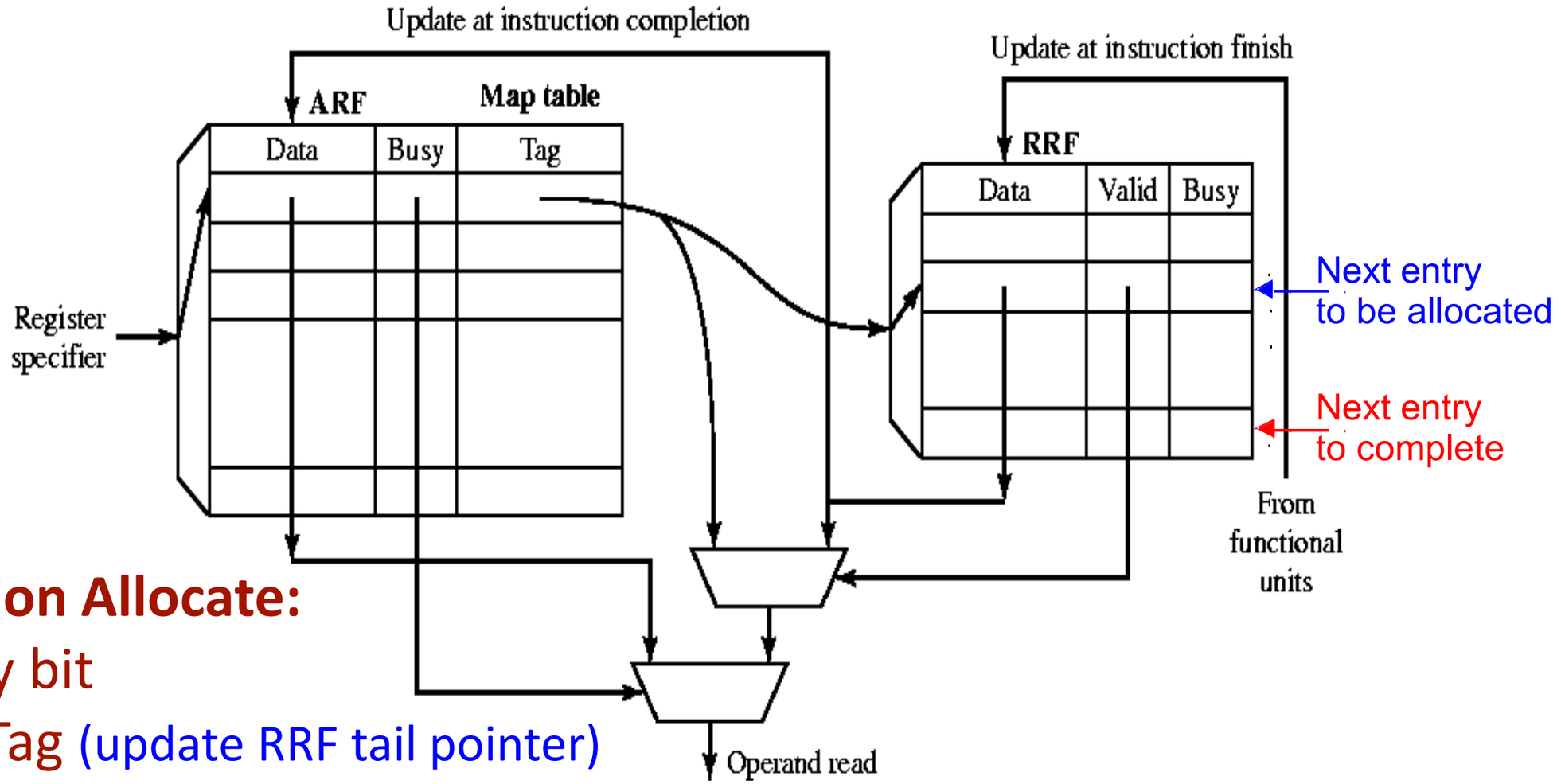
a. Busy = 0 → no pending W

b. Busy = 1 → yes pending W

i. Valid = 1 → update finished

ii. Valid = 0 → update not done yet

Register Renaming Tasks – Destination Allocate



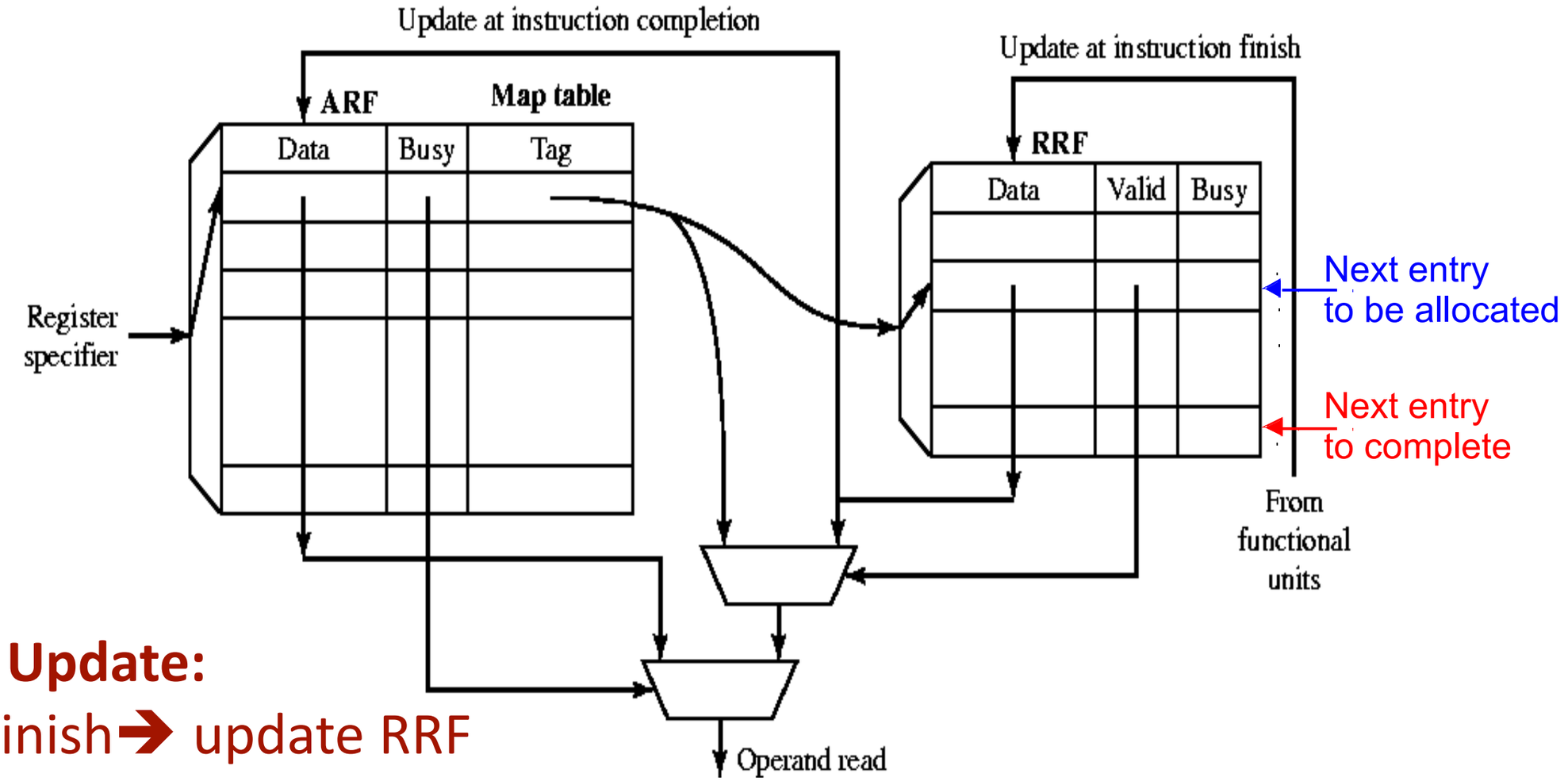
2. Destination Allocate:

a. Set Busy bit

b. Assign Tag (update RRF tail pointer)

c. Update Map Table

Register Renaming Tasks – Register Update



3. Register Update:

a. when Finish➔ update RRF

b. when Complete ➔ copy to ARF (update RRF head pointer)

Register Renaming Difficulties:

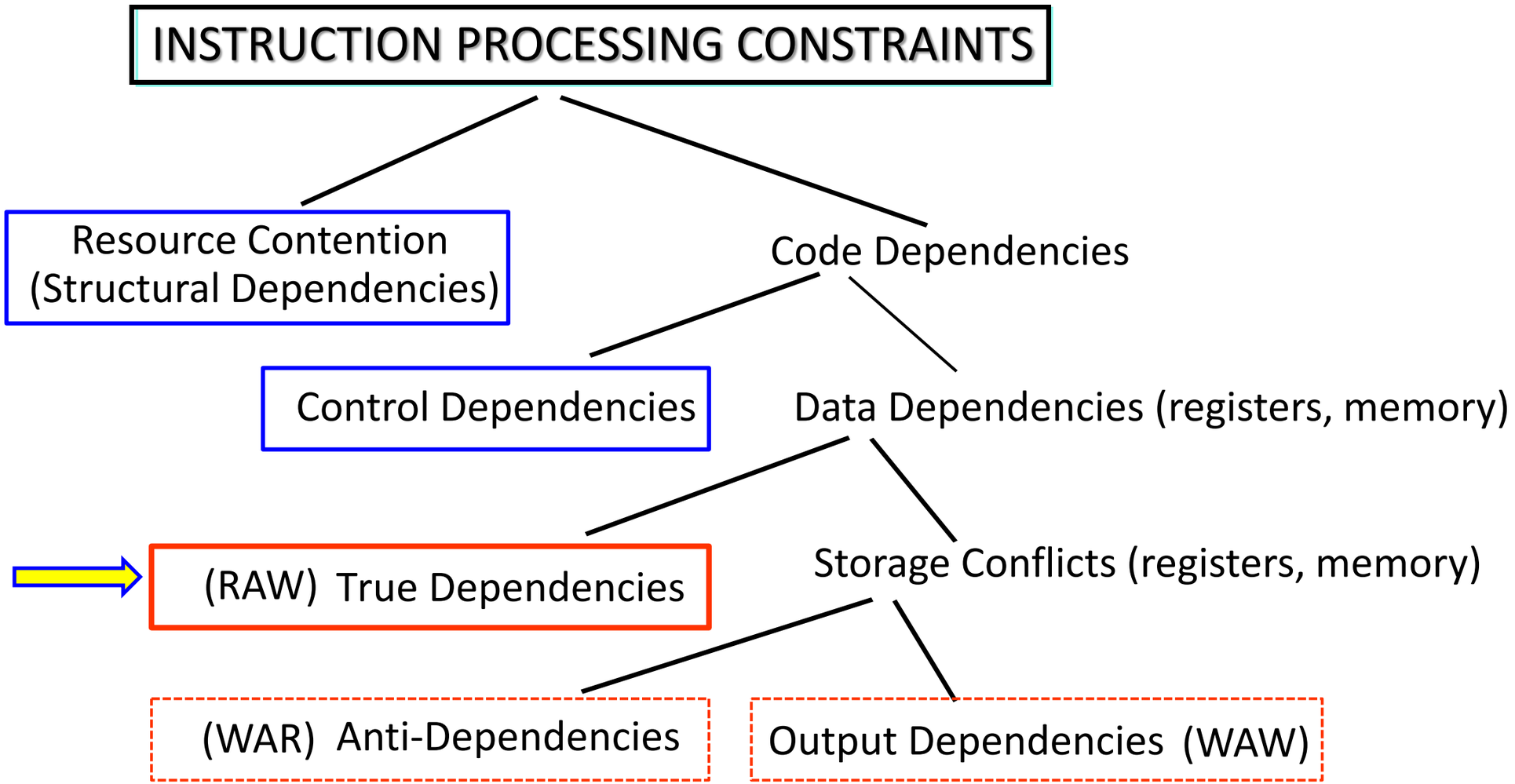
Wide Instruction Decode/Dispatch:

- Need many ports in RFs and mapping tables
- Instruction dependencies during dispatching, issuing, committing
  - Must handle dependencies across instructions
  - Implementation: use comparators, multiplexors, counters
    - Comparators: discover RAW dependencies
    - Multiplexors: generate right physical address (old or new allocation)
    - Counters: determine number of physical registers allocated

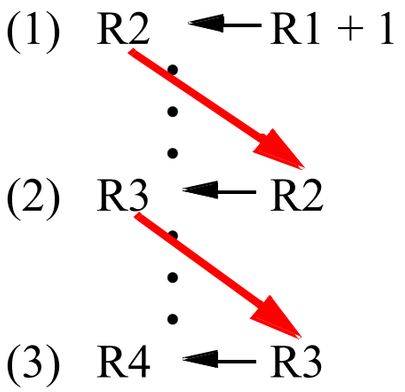
On Mispredictions/Exceptions:

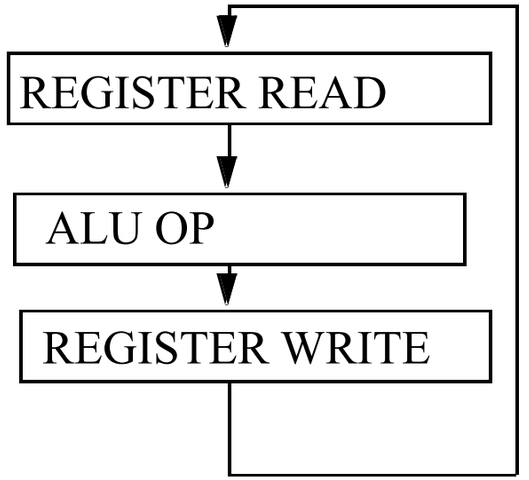
- Register mapping must be precise, i.e. restored precisely
  - Unroll register renaming back to point of misprediction/exception

The Big Picture: Impediments Limiting ILP



Resolving True Data Dependencies





STALL DISPATCHING

ADVANCE INSTRUCTIONS

"DYNAMIC EXECUTION"

1) Read register(s), get "IOU" if not ready
2) Advance to reservation station
3) Wait for "IOU" to show up
4) Execute

Reservation Station + Complex Forwarding

Out-of-order (OoO) Execution

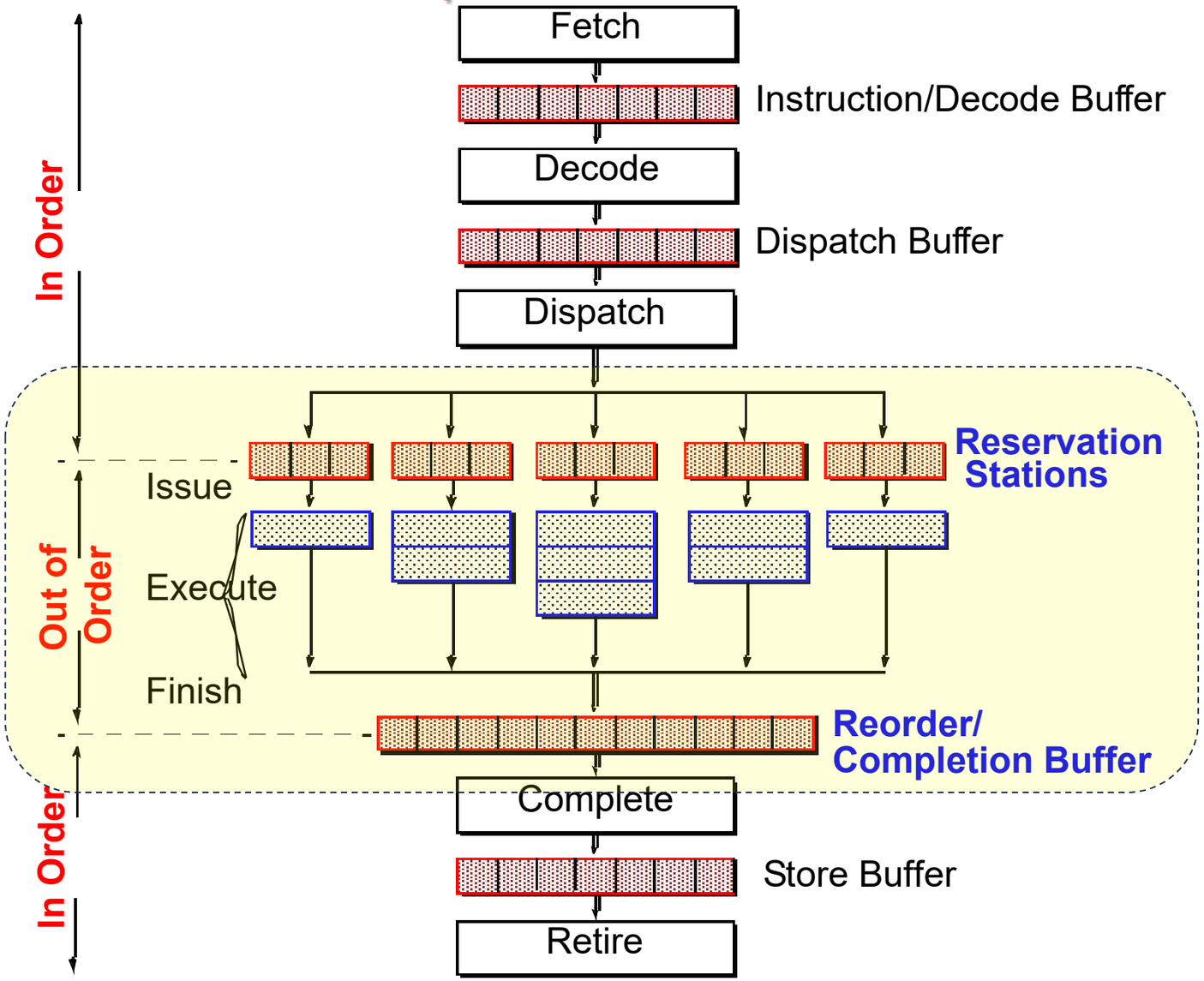
Try to Approach the "Data-Flow Limit"

B. Out-of-Order Instruction Execution



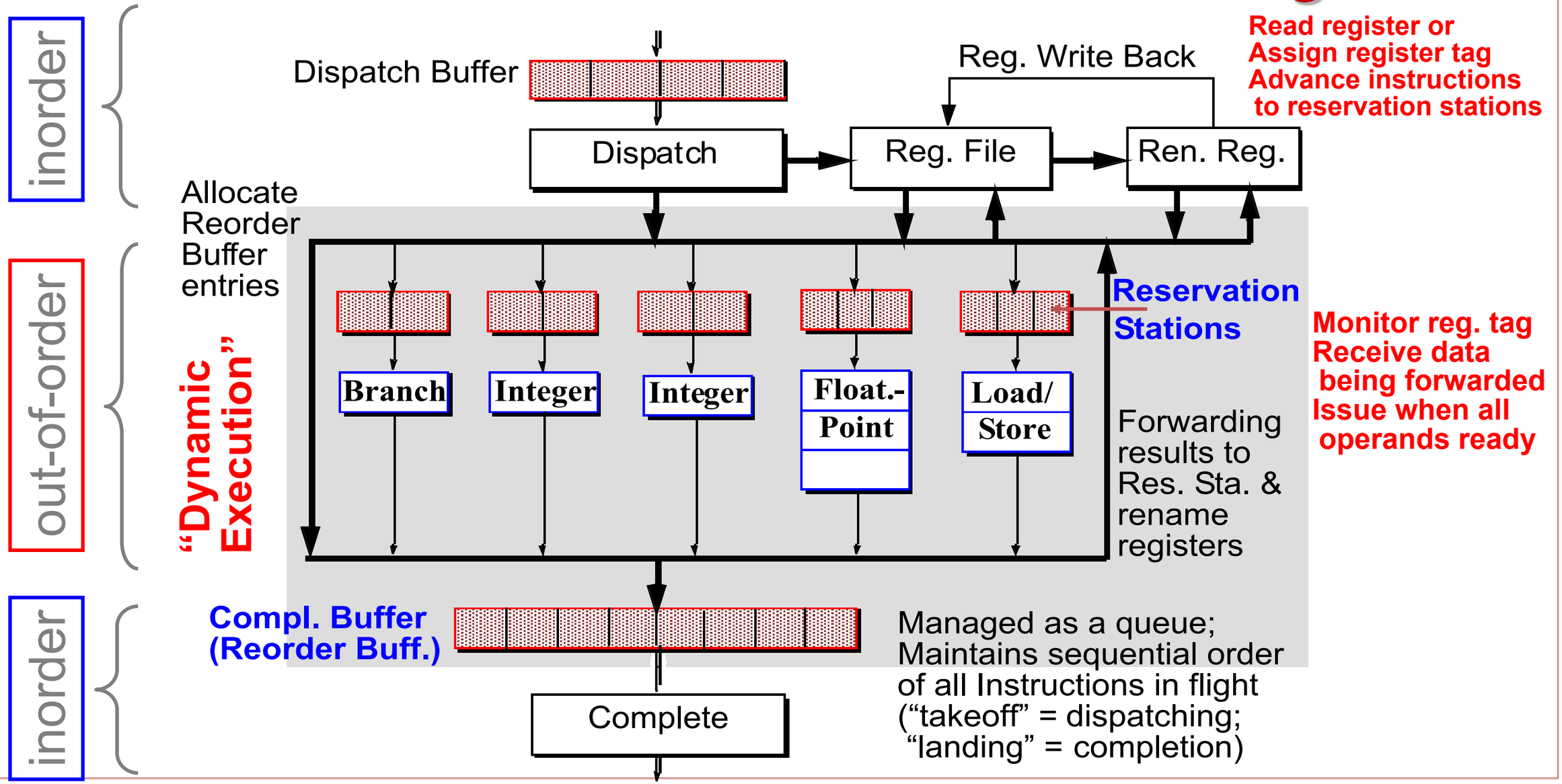
Electrical & Computer ENGINEERING

Modern Superscalar Processor Organization



- Buffers provide decoupling
- In OOO designs they also facilitate (re-)ordering
- More details on specific buffers to follow

Elements of Modern Micro-Dataflow Engine



Steps in Dynamic OOO Execution (1)

- FETCH instruction (in-order, speculative)
  - I-cache access, branch predictions , BTB, PHT, etc.
  - Can also access from trace cache, loop buffer, etc.
- DISPATCH (in-order, speculative)
  - Read operands from Register File (ARF) and/or Rename Register File (RRF)
    - RRF may return a ready value or a Tag for a physical location
  - Allocate new RRF entry (rename destination register) for destination
  - Allocate and update **Reorder Buffer** (ROB) entries
  - Advance instruction to appropriate entry in the scheduling hardware
    - Typical name for centralized: **Issue Queue** or **Instruction Window**
    - Typical name for distributed: **Reservation Stations**

Steps in Dynamic OOO Execution (2)

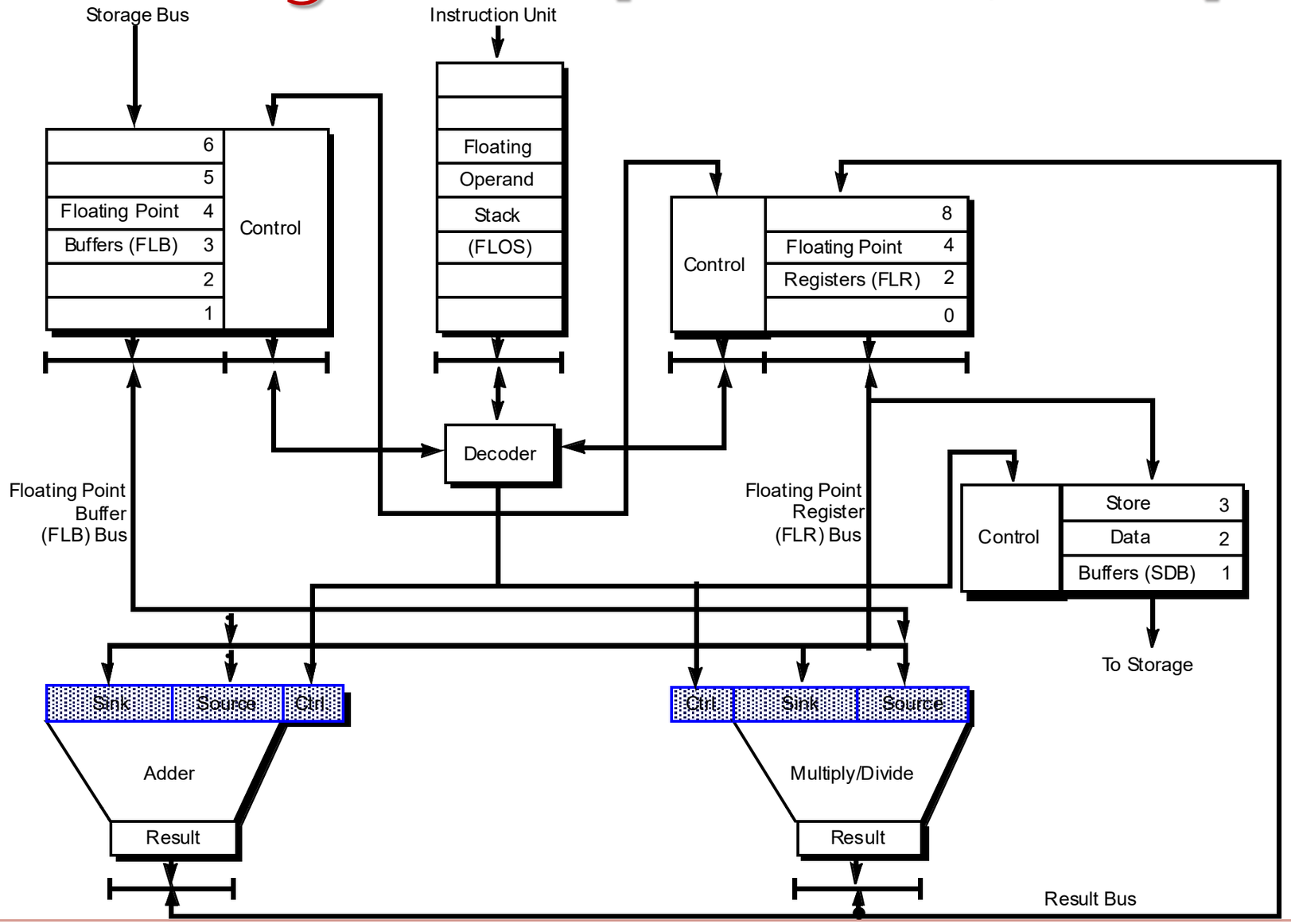
- ISSUE & EXECUTE & FINISH (out-of-order, speculative)
  - Scheduler entry monitors result bus for rename register Tag(s) for pending operand(s)
    - Find out if source operand becomes ready; if Tag(s) match, latch in operand(s)
  - When all operands ready, instruction is ready to be issued into FU (wake-up)
  - Issue instruction into FU, de-allocate scheduler entry, no further stalling in FU pipe
    - Issuing is subject to structural hazards and scheduling priorities (select)
  - When execution finishes, broadcast result to waiting scheduler entries and RRF entry
- COMPLETE/COMMIT/RETIRE/GRADUATE (in-order, non-speculative)
  - When ready to commit result into "in-order" (architectural) state (head of the ROB):
    - Update architectural register from RRF entry, de-allocate RRF entry, and if it is a store instruction, advance it to Store Buffer
    - De-allocate ROB entry and instruction is considered architecturally completed
    - Update predictors based on instruction result

Some Historical Background

- Dynamic or Data-Flow Scheduling: "**Data-Flow Limit**"
  - Scheduling hardware that allows each instruction to be executed as soon as its source operands are ready and a FU is available
  - Assuming register renaming, only limited by RAW and structural hazards
- First proposal: Tomasulo's algorithm in IBM 360/91 FPU (1967)
  - 1 instruction per cycle, distributed implementation, imprecise exceptions…
- We will get to modern out-of-order implementations, but first …
  - Let's cover the historical Tomasulo's algorithm that started it all
  - Modern Differences: all instruction types, register renaming, precise exceptions, multiple instructions per cycle, aggressive speculations, etc.

C. Tomasulo's Algorithm (historical reference)

Tomasulo's Algorithm [Tomasulo, 1967]



IBM 360/91 FPU

- **Multiple functional units (FU's)**
  - Floating-point add
  - Floating-point multiply/divide
- **Three register files (pseudo reg-reg machine in floating-point unit)**
  - (4) floating-point registers (FLR)
  - (6) floating-point buffers (FLB)
  - (3) store data buffers (SDB)
- **Out of order instruction execution**:
  - After decode the instruction unit passes all floating point instructions (in order) to the floating-point operation stack (FLOS) [actually a queue, not a stack]
  - In the floating point unit, instructions are then further decoded and issued from the FLOS to the two FU's
- **Variable operation latencies**:
  - Floating-point add: 2 cycles
  - Floating-point multiply: 3 cycles
  - Floating-point divide: 12 cycles
- Goal: **achieve concurrent execution of multiple floating-point instructions, in addition to achieving one instruction per cycle in instruction pipeline**

Dependency Mechanisms

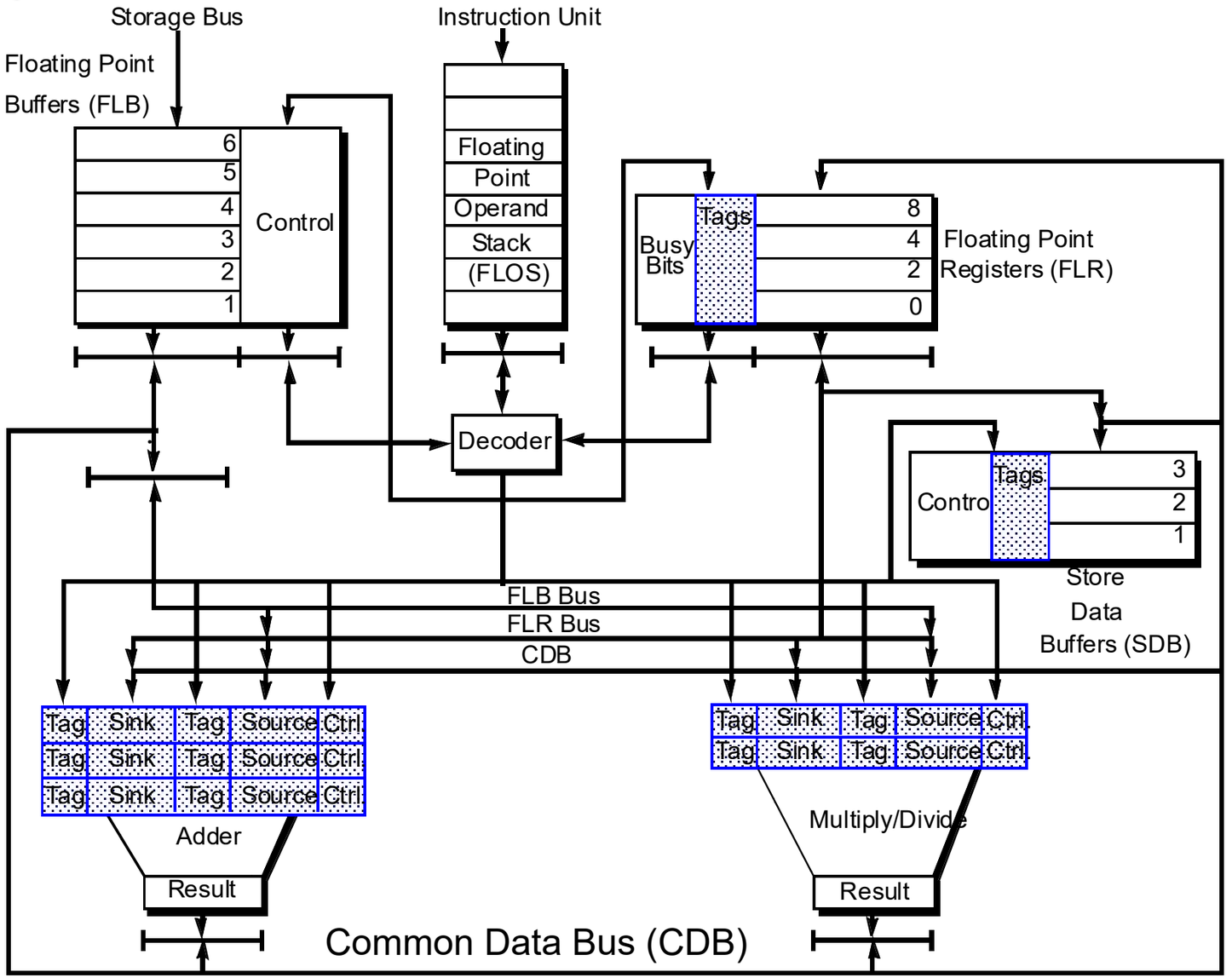
Two Address IBM 360 Instruction Format:

R1 <-- R1 op R2

Major dependency mechanisms:

- **Structural (FU) dependency = > virtual FU's**
  - Reservation stations
- **True dependency = > pseudo operands + result forwarding**
  - Register tags
  - Reservation stations
  - Common data bus (CDB)
- **Anti-dependency = > operand copying**
  - Reservation stations
- **Output dependency = > register renaming + result forwarding**
  - Register tags
  - Reservation stations
  - Common data bus (CDB)

IBM 360/91 FPU



Reservation Stations

- Used to collect operands or pseudo operands (tags).
- Associate more than one set of buffering registers (control, source, sink) with each FU, → virtual FU's.
- Add unit: three reservation stations
- Multiply/Divide unit: two reservation stations

Tag	Sink	Tag	Source
0 implies valid data	Source value	0 implies valid data	Source value

Common Data Bus (CDB)

- CDB is fed by all units that can alter a register (or supply register values) and it feeds all units which can have a register as an operand.
- Sources of CDB:
  - Floating-point buffers (FLB)
  - Two FU's (add unit and the multiply/divide unit)
  - 6 FLB + 3 addRS + 2 muldivRS = 11 unique sources
  - 3 physical sources (FLB, adder, mul/div)
- Destinations of CDB:
  - Reservation stations
  - Floating-point registers (FLR)
  - Store data buffers (SDB)
  - (5 RS x 2) + 4 FLR + 3 SDB : CDB has 17 destinations
- Electrically very challenging
  - 3 physical sources must arbitrate for access to CDB
  - Tag + data must be driven to 17 destinations

Register Tags

- **Every source of a register value must be uniquely identified by its own tag value.**
  - (6) FLB's
  - (5) reservation stations (3 with add unit, 2 with multiply/divide unit)

    ➔ 4-bit tag is needed to identify the 11 potential sources

- **Every destination of a register value must carry a tag field**.
  - (5) "sink" entries of the reservation stations
  - (5) "source" entries of the reservation stations
  - (4) FLR's
  - (3) SDB's

    ➔ a total of 17 tag fields are needed (i.e. 17 places that need tags)

Operation of Dependency Mechanisms

1. **Structural (FU) dependency ➔ virtual FU's**
   - FLOS can hold and decode up to 8 instructions.
   - Instructions are dispatched to the 5 reservation stations (virtual FU's) even though there are only two physical FU's.
   - Hence, structural dependency does not stall dispatching.

2. **True dependency ➔ pseudo operands + result forwarding**
   - If an operand is available in FLR, it is copied to a res. station entry.
   - If an operand is not available (i.e. there is pending write), then a tag is copied to the res. station entry instead. This tag identifies the source of the pending write. This instruction then waits in its reservation station for the true dependency to be resolved.
   - When the operand is finally produced by the source (ID of source = tag value), this source asserts its ID, i.e. tag value, on the CDB followed by broadcasting the operand on the CDB.
   - All the reservation station entries and the FLR entries and SDB entries carrying this tag value in their tag fields will detect a match of tag values and latch in the broadcasted operand from the CDB.
   - Hence, true dependency does not block subsequent independent instructions and does not stall a physical FU. Forwarding also minimizes delay due to true dependency.

Example 1

i: R2 <= R0 + R4
j: R8 <= R0 + R2 (RAW on R2)

Cycle #1

DISPATCHED INSTRUCTION(S): i

	ID	Tag	Sink	Tag	Source
i	1	0	6.0	0	10.0
	2				
	3				
	Adder		i		

ID	Tag	Sink	Tag	Source
4				
5				
Mult/Div				

	Busy	Tag	Data
0			6.0
2	x	1	3.5
4			10.0
8			7.8

Cycle #2

DISPATCHED INSTRUCTION(S): j

	ID	Tag	Sink	Tag	Source
i	1	0	6.0	0	10.0
j	2	0	6.0	1	16.0
	3				
	Adder		i		

ID	Tag	Sink	Tag	Source
4				
5				
Mult/Div				

	Busy	Tag	Data
0			6.0
2	x	1	16.0
4			10.0
8	x	2	7.8

Cycle #3

DISPATCHED INSTRUCTION(S):

	ID	Tag	Sink	Tag	Source
	1				
j	2	0	6.0	0	16.0
	3				
	Adder		j		

ID	Tag	Sink	Tag	Source
4				
5				
Mult/Div				

	Busy	Tag	Data
0			6.0
2			16.0
4			10.0
8	x	2	7.8

Operation of Dependency Mechanisms

3. **Anti-dependency** ➔ **operand copying**
   - If an operand is available in FLR, it is copied to a reservation station entry.
   - By copying this operand to the reservation station, all anti-dependencies due to future writes to this same register are resolved.
   - Hence, the reading of an operand is not delayed, possibly due to other dependencies, and subsequent writes are also not delayed.

Example 2

i: R4 <= R0 * R8
j: R0 <= R4 * R2 (RAW on R4)
k: R2 <= R2 + R8 (WAR on R2)

Cycle #1

DISPATCHED INSTRUCTION(S): i, j

ID	Tag	Sink	Tag	Source
1				
2				
3				
Adder				

	ID	Tag	Sink	Tag	Source
i	4	0	6.0	0	7.8
j	5	4		0	3.5
Mult/Div			i		

	Busy	Tag	Data
0	x	5	6.0
2			3.5
4	x	4	10.0
8			7.8

Cycle #2

DISPATCHED INSTRUCTION(S): k

	ID	Tag	Sink	Tag	Source
k	1	0	3.5	0	7.8
	2				
	3				
Adder		k			

	ID	Tag	Sink	Tag	Source
i	4	0	6.0	0	7.8
j	5	4		0	3.5
Mult/Div			i		

	Busy	Tag	Data
0	x	5	6.0
2	x	1	3.5
4	x	4	10.0
8			7.8

Cycle #3

DISPATCHED INSTRUCTION(S):

	ID	Tag	Sink	Tag	Source
k	1	0	3.5	0	7.8
	2				
	3				
Adder		k			

	ID	Tag	Sink	Tag	Source
i	4	0	6.0	0	7.8
j	5	4		0	3.5
Mult/Div			i		

	Busy	Tag	Data
0	x	5	6.0
2	x	1	3.5
4	x	4	10.0
8			7.8

Operation of Dependency Mechanisms

3. Output dependency ➔ register renaming + result forwarding

- If a register is waiting for a pending write, its tag field will contain the ID, or tag value, of the source for that pending write.
- When that source eventually produces the result, that result will be written into the register via the CDB.
- It is possible that prior to the completion of the pending write, another instruction can come along and also has that same register as its destination register.
- If this occurs, the operands (or pseudo operands) needed by this instruction are still copied to an available reservation station. In addition, the tag field of the destination register of this instruction is updated with the ID of this new reservation station, i.e. the old tag value is overwritten. This will ensure that the said register will get the latest value, i.e. the late completing earlier write cannot overwrite a later write.
- Hence, the output dependency is resolved without stalling a physical functional unit, not requiring additional buffers to ensure sequential write back to the register file.

Example 3

What if j causes FP overflow exception?
- where is R4?
- it is lost => imprecise exceptions!

i: R4 <= R0 * R8
j: R2 <= R0 + R4 (RAW on R4)
k: R4 <= R0 + R8 (WAW on R4)
l: R8 <= R4 * R8 (RAW on R4)

Cycle #1

DISPATCHED INSTRUCTION(S): i, j

	ID	Tag	Sink	Tag	Source
j	1	0	6.0	4	
	2				
	3				
	Adder				

	ID	Tag	Sink	Tag	Source
i	4	0	6.0	0	7.8
	5				
	Mult/Div		i		

	Busy	Tag	Data
0			6.0
2	x	1	3.5
4	x	4	10.0
8			7.8

Cycle #2

DISPATCHED INSTRUCTION(S): k, l

	ID	Tag	Sink	Tag	Source
j	1	0	6.0	4	
k	2	0	6.0	0	7.8
	3				
	Adder		k		

	ID	Tag	Sink	Tag	Source
i	4	0	6.0	0	7.8
l	5	2		0	7.8
	Mult/Div		i		

	Busy	Tag	Data
0			6.0
2	x	1	3.5
4	x	2	10.0
8	x	5	7.8

Cycle #3

DISPATCHED INSTRUCTION(S):

	ID	Tag	Sink	Tag	Source
j	1	0	6.0	4	
k	2	0	6.0	0	7.8
	3				
	Adder		k		

	ID	Tag	Sink	Tag	Source
i	4	0	6.0	0	7.8
l	5	2		0	7.8
	Mult/Div		i		

	Busy	Tag	Data
0			6.0
2	x	1	3.5
4	x	2	13.8
8			7.8

Summary of Tomasulo's Algorithm

- Supports out of order execution of instructions.
- Resolves dependencies dynamically using hardware.
- Attempts to delay the resolution of dependencies as late as possible.
- Structural dependency does not stall issuing; virtual FU's in the form of reservation stations are used.
- Output dependency does not stall issuing; copying of old tag to reservation station and updating of tag field of the register with pending write with the new tag.
- True dependency with a pending write operand does not stall the reading of operands; pseudo operand (tag) is copied to reservation station.
- Anti-dependency does not stall write back; earlier copying of operand awaiting read to the reservation station.
- Can support sequence of multiple output dependencies.
- Forwarding from FU's to reservation stations bypasses the register file.

Tomasulo vs. Modern OOO

	IBM 360/91	Modern Superscalar
Width	Peak IPC = 1	Peak IPC = 8-10
Structural hazards	2 FPU Single CDB	Many FU Many busses
Anti-dependencies	Operand copy	**Reg. Renaming**
Output dependencies	Renamed reg. tag	**Reg. renaming**
True dependencies	Tag-based forwarding	Tag-based forwarding
Exceptions	**Imprecise**	Precise (ROB)
Implementation	3 x 66” x 15” x 78” 60ns cycle time 11-12 gate delays per pipe stage > $1 million	1 ARM CPU core << 10mm^2 @3nm ~ 200ps @5 GHz < $10 (?)

18-740: Modern Computer Architecture & Design

Lecture 6:
"Dynamic Instruction Scheduling Techniques"

John Paul Shen
September 10, 2026

Next Time ...

- Required Reading Assignments:
  - **Chapter 5 and Chapter 7 of Shen and Lipasti (SnL).**
- Recommended Reference:
  - http://en.Wikipedia.org/wiki/Pentium_Pro

Electrical & Computer ENGINEERING